Chapter 2: Programs and Assignments

Chapter 1 gave Vertex enough structure to evaluate one arithmetic expression. We could run:

```
3+4*5
```

and obtain `23`. The interpreter did not remember anything after producing that value, and Vertex itself could not print a result. Python printed the value returned by the evaluator.

This chapter adds a small program:

```
x = 2;
print x;
```

The program creates a variable, stores a value under its name, and retrieves that value later. These two statements must execute in order, and the result of the first statement must remain available to the second. That requires three connected additions to the language:

- programs made from statements
- variables and assignment
- an environment that preserves state while the program runs

The Vertex interpreter in chapter-02-programs-and-assignments/code implements assignment statements and print statements. The expression language from Simple Expressions still works, including floating-point literals and unary minus. Expressions can now also contain identifiers. Assignment remains a statement in this chapter. It is not part of the expression grammar.

This chapter follows runner.py, tokenizer.py, parser.py, and evaluator.py. Code excerpts come from those files, with occasional formatting adjustments.

Running a Program

From this chapter's code directory, run the supplied program:

```
python3 runner.py example.v
```

Its source is:

```
// Assign a value, then look it up for printing.
x = 2; // The semicolon separates this assignment from the next statement.
print x;
// Multiplication happens before addition, so y becomes 14.
y = x + 3 * 4;
print y; // Output: 14.
```

The output is:

```
2
14
```

The runner also accepts a complete program as one command-line argument:

```
python3 runner.py 'x = 2; print x'
```

Quoting keeps the source together as one shell argument. When the argument ends in `.v` or `.t`, the runner reads the source from that file instead. The `.v` extension identifies a Vertex source file. Support for `.t` remains for compatibility with earlier material.

Unlike Chapter 1, the runner does not print the value returned by the evaluator. Vertex now has its own print statement. Assignment and print perform their work and return `None` to the Python runner.

From Expressions to Programs

An expression represents a computation that produces a value. A statement represents an action performed while a program runs. (1)

In this program:

```
x = 2;
print x;
```

the expression `2` produces a value. The assignment statement associates that value with the name `x`. (2) In the second statement, the expression `x` retrieves the stored value. The print statement sends that value to the output.

The distinction matters because the interpreter now has more than one kind of result to track.

Construct	Purpose	Example
Expression	Produce a value	`x + 3 * 4`
Assignment statement	Change the environment	`x = 2`
Print statement	Produce output	`print x`
Statement list	Execute statements in order	`x = 2; print x`
Program	Supply the complete unit of execution	the source file

A program is no longer one expression with a final numeric answer. Its behavior includes changes to state and visible output.

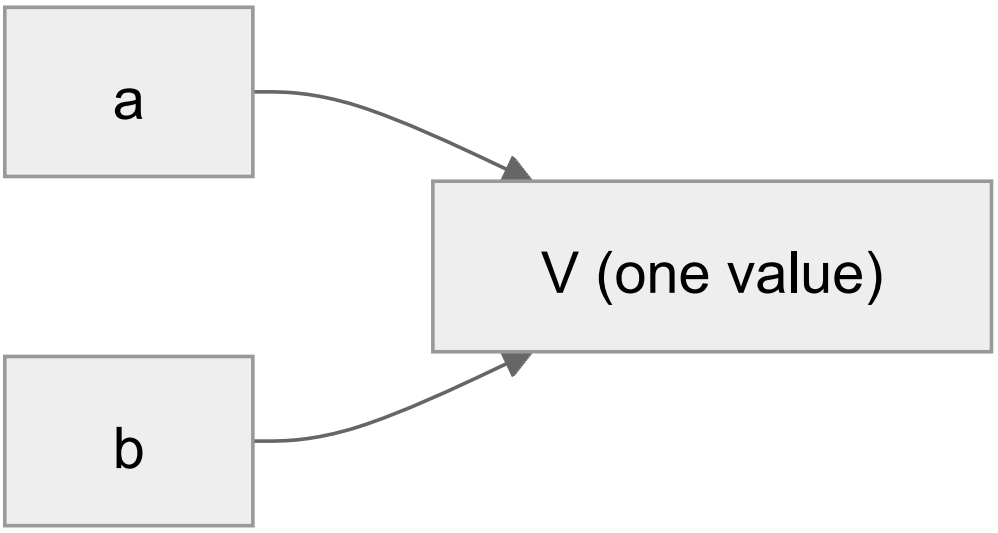
Identifiers Refer to Values

An identifier and the value it refers to are different things. Assignment establishes a **binding** between them. Evaluating an identifier retrieves its value; it does not retrieve another name that must be followed later.

Suppose `V` labels a value. This is a conceptual trace, not Vertex source:

```
a = V
b = a
```

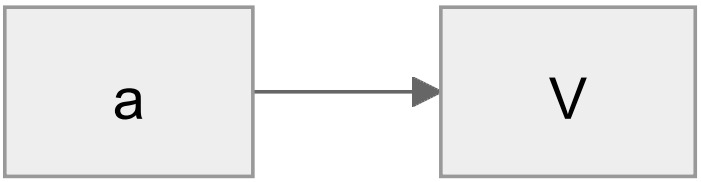
The right side of `b = a` evaluates to `V`, because that is the value of `a`. Now both identifiers point to `V`. They are **aliases** for the same value. (3) Assignment copies the reference; it does not duplicate the value.

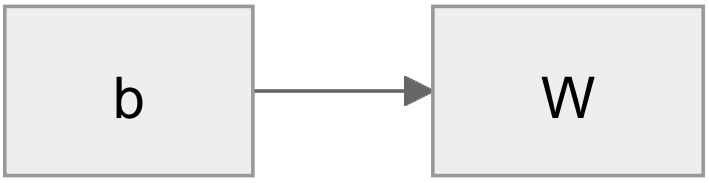


Now suppose we assign a different value, `W`, to `b`:

```
b = W
```

This **rebinds** `b`. It changes what `b` points to. It does not change `V`, and `a` still points to `V`:





First, give concrete values to the `b = W` diagram above:

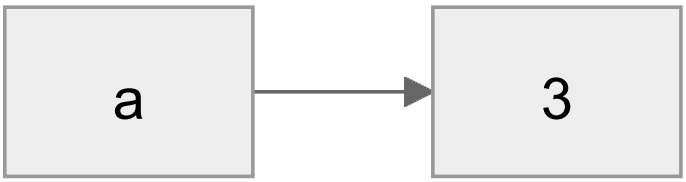
```
a = 2;
b = a;
b = 7;
print a;
print b;
```

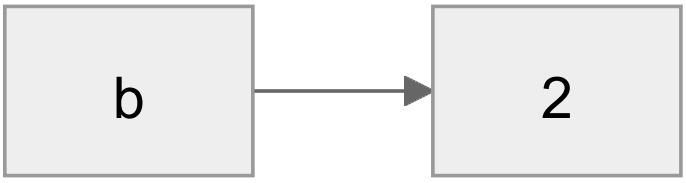
This prints `2`, then `7`: `a` still reaches `2`, while `b` now reaches `7`.

Now reset the bindings and rebind `a` instead. This is the mirror case:

```
a = 2;
b = a;
a = 3;
print a;
print b;
```

The output is `3`, then `2`. `b` refers to the value `2`, not to the identifier `a`. Rebinding `a` does not change that value or `b`'s reference:





Likewise, `a = a + 1` computes a new number and rebinds `a`; it does not change the contents of the old number.

Go back to the state after `b = a`, when both identifiers point to `V`. Suppose we could change something *inside* `V` without replacing it. Then both aliases would see that change. Every identifier pointing to `V` would still reach that same, now modified value. That is **mutation**, and it is different from rebinding an identifier.

Numbers are **immutable**: there is no operation that changes something inside the number `2`. Aliasing already happens, but there is no internal change for the aliases to observe. When we reach complex values whose parts can change, sharing a value will have visible consequences. The assignment rule will stay the same.

The Topic 2 Grammar

The parser records the grammar in Extended Backus-Naur Form (EBNF). (4)

```
program ::= statement_list

statement_list ::= { ";" } statement
                   { ";" { ";" } statement }
                   { ";" }

statement ::= assignment_statement | print_statement
assignment_statement ::= <identifier> "=" expression
print_statement ::= "print" expression

expression ::= term { ("+" | "-") term }
term ::= unary { ("*" | "/") unary }
unary ::= "-" unary | factor
factor ::= <number> | <identifier> | "(" expression ")"
```

The expression grammar is the one from Simple Expressions with identifiers added as a new factor form. That lets expressions refer to values stored earlier in the program, as in `x`, `x+1`, and `3*(x+5)`.

The new rules sit above expressions. An assignment statement begins with an identifier and an equals sign. A print statement begins with the keyword `print`. A statement is one of those two forms. A statement list contains one or more statements, and a program contains a statement list.

The grammar deliberately does not include assignment as an expression. For example, this is not a valid Topic 2 program:

```
print (x = 2)
```

Supporting assignment expressions requires additional parsing decisions. This chapter keeps assignment at the statement level so that the new ideas remain separate.

Statement Lists and Semicolons

The statement-list rule permits extra semicolons at the beginning and end. It requires at least one semicolon between adjacent statements.

Source	Valid?	Reason
`x = 2`	Yes	One statement needs no bracket or terminator
`x = 2;`	Yes	A trailing semicolon is allowed
`;;;x = 2;;;`	Yes	Leading and trailing semicolons are allowed
`x = 2; print x`	Yes	One semicolon separates the statements
`x = 2;;; print x`	Yes	More than one separator is allowed
`x = 2 print x`	No	Adjacent statements require a semicolon
empty input	No	A statement list contains at least one statement

The semicolons do not create a compound statement or block. Topic 2 has no braces and no nested statement lists. The program simply contains one top-level list of statements.

Newlines count as whitespace. They make a file easier to read, but they do not replace semicolons. These two programs therefore have the same structure:

```
x = 2; print x
```

```
x = 2;
print x
```

The tokenizer discards `//` comments just as it does in Simple Expressions. A newline ending a comment still does not separate statements:

```
x = 2; // Keep the semicolon before the comment.
print x
```

Removing that semicolon makes the program invalid. Moving it after `//` does not help: it becomes part of the discarded comment, leaving no separator between the assignment and `print`.

Recognizing the New Tokens

The tokenizer adds patterns for assignment, semicolons, the print keyword, and identifiers:

```
(r"\=", "="),
(r"\;", ";"),
(r"print\b", "print"),
(r"[a-zA-Z_][\w]*", "identifier"),
```

The identifier pattern requires a letter or underscore first. Later characters may also include digits. Examples include `x`, `total`, `_temporary`, and `item2`.

The `print` pattern appears before the identifier pattern. The first matching pattern wins, so the source word `print` receives the tag `print` rather than `identifier`. The `\b` requires a word boundary. It prevents the tokenizer from splitting `printer` into the keyword `print` followed by the identifier `er`.

The number and operator rules carry forward from Simple Expressions. Topic 2 adds names and statement punctuation around that expression tokenizer.

Tokenizing `x = 2; print x` produces tokens equivalent to:

```
[
    {"tag": "identifier", "value": "x"},
    {"tag": "="},
    {"tag": "number", "value": 2},
    {"tag": ";"},
    {"tag": "print"},
    {"tag": "identifier", "value": "x"},
    {"tag": None},
]
```

The actual tokens also carry line and column positions. Whitespace and comments do not become tokens, but the tokenizer still counts their characters when updating those positions.

The Program Tree

The parser produces a tree for the entire program. For `x = 2; print x`, the tree is:

```
{
    "tag": "program",
    "statements": {
        "tag": "statement_list",
        "statements": [
            {
                "tag": "assign",
                "target": {"tag": "identifier", "value": "x"},
                "expression": {"tag": "number", "value": 2},
            },
            {
                "tag": "print",
                "expression": {"tag": "identifier", "value": "x"},
            },
        ],
    },
}
```

The program and statement-list nodes preserve the sequence. Their statements remain in source order. The assignment node keeps an identifier destination separately from the expression that supplies its value. The print node contains the expression whose value will be printed.

Semicolons do not appear as AST nodes. They tell the parser where statements are separated, but the evaluator does not need to execute punctuation. Whitespace and the `print` token also disappear as separate pieces once their meaning has been captured by the tree.

Parsing Assignment and Print

The assignment parser expects an identifier, an equals sign, and an expression:

```
def parse_assignment_statement(tokens):
    assert tokens[0]["tag"] == "identifier", "Expected <identifier>"
    identifier = {"tag": "identifier", "value": tokens[0]["value"]}
    tokens = tokens[1:]
    assert tokens[0]["tag"] == "=", "Expected '=' for assignment"
    tokens = tokens[1:]
    ast, tokens = parse_expression(tokens)
    return {"tag": "assign", "target": identifier,
            "expression": ast}, tokens
```

The target is an identifier node, not a string literal or a bare name. In this position it creates a destination for the value. The evaluator does not look up an old value for the target, so assigning to a new identifier works. The name inside that node is still the dictionary key used to store the binding.

The right side becomes an ordinary expression tree. Arithmetic precedence therefore works inside an assignment without any new arithmetic rules.

The print parser consumes the keyword and parses the following expression:

```
def parse_print_statement(tokens):
    assert tokens[0]["tag"] == "print", "Expected 'print'"
    tokens = tokens[1:]
    ast, tokens = parse_expression(tokens)
    return {"tag": "print", "expression": ast}, tokens
```

The statement parser chooses between the two forms by inspecting the first token:

```
def parse_statement(tokens):
    if tokens[0]["tag"] == "print":
        return parse_print_statement(tokens)
    if tokens[0]["tag"] == "identifier":
        return parse_assignment_statement(tokens)
    raise SyntaxError(f"Expected statement, got {tokens[0]}")
```

This decision is simple because the two statements begin differently. Topic 2 does not need to parse an expression first and later decide whether it was an assignment.

Parsing a Statement List

The statement-list parser first consumes any leading semicolons and then requires its first statement:

```
statements = []

while tokens[0]["tag"] == ";":
    tokens = tokens[1:]

statement, tokens = parse_statement(tokens)
statements.append(statement)
```

After that, another statement can begin only after at least one semicolon. A run of semicolons is consumed together:

```
while True:
    if tokens[0]["tag"] != ";":
        break

    while tokens[0]["tag"] == ";":
        tokens = tokens[1:]

    if tokens[0]["tag"] is None:
        break

    statement, tokens = parse_statement(tokens)
    statements.append(statement)
```

If the semicolons reach the end marker, they were trailing semicolons. Otherwise, the parser requires another statement. Finally, it returns a statement-list node and the unused tokens:

```
return {"tag": "statement_list", "statements": statements}, tokens
```

Requiring the first statement makes an empty program invalid. Checking for a semicolon before each later statement enforces the separator rule. The public parser still verifies that the entire token stream has been consumed, so `x = 2 print x` cannot be accepted as a shorter program with ignored input.

The Environment

An environment records the values currently associated with names. The Topic 2 interpreter represents it with a Python dictionary:

```
{}
```

Before the program runs, the environment is empty. After executing `x = 2`, it is:

```
{"x": 2}
```

After executing `y = x + 3 * 4`, it is:

```
{"x": 2, "y": 14}
```

The source name `x` is a Vertex identifier. The string key `"x"` is how the Python implementation represents that name inside the environment. The stored Python number represents the current Vertex value.

An environment is state because its contents can change while the program executes. Evaluating the same identifier expression can therefore produce different values at different times:

```
x = 2;
print x;
x = 7;
print x;
```

The output is `2` followed by `7`. The identifier expression has the same source text both times, but assignment has changed the environment between the two evaluations.

Evaluating Identifiers and Assignment

An identifier node asks the environment for the value associated with its name. If the name cannot be found, evaluation raises an error:

```
elif ast["tag"] == "identifier":
    identifier = ast["value"]
    env = environment
    while True:
        if identifier in env:
            return env[identifier]
        if "$PARENT" in env:
            env = env["$PARENT"]
            continue
        raise ValueError(f"Unknown identifier: {identifier}")
```

The direct lookup in the current dictionary is all that Programs and Assignments requires. The code also supports a **parent environment**: an enclosing environment that can supply bindings missing from the current one. The special Python dictionary key `$PARENT` refers to that dictionary; it is not a Vertex variable name. No syntax in this chapter creates a parent environment, but we can construct one in Python:

```
parent = {"x": 10, "y": 20}
environment = {"$PARENT": parent, "x": 3}
```

Lookup checks the current environment first. If the name is missing, it follows `$PARENT` and checks there, repeating until it finds the name or reaches an environment with no parent. The first matching binding wins. Reaching the end without a match raises an unknown-identifier error.

Name	Search	Result
`x`	Found in the current environment	`3`, not the parent's `10`
`y`	Missing here; found in the parent	`20`
`z`	Missing in both; no further parent	Error

Assignment works differently: it writes to the **current environment**, without searching the parent chain for a destination. Evaluating `y = 7` in this example creates a local `y` binding. Later lookup returns `7`, while the parent's `y` remains `20`. We will return to nested environments when we study scope.

Assignment evaluates its expression first, then stores the result under the target name:

```
elif ast["tag"] == "assign":
    value = evaluate(ast["expression"], environment)
    target = ast["target"]
    if target["tag"] != "identifier":
        raise ValueError("Assignment requires an identifier destination")
    environment[target["value"]] = value
    return None
```

The order matters in a reassignment such as:

```
x = x + 1
```

The evaluator reads the old value of `x` while evaluating the right side. It then replaces the binding with the new value. Assignment returns `None` because the grammar treats it as a statement rather than an expression.

Evaluating Print and Programs

The print case evaluates its expression in the current environment and sends the result to Python's print function:

```
elif ast["tag"] == "print":
    result = evaluate(ast["expression"], environment)
    print(result)
    return None
```

The statement-list case evaluates every statement in order:

```
elif ast["tag"] == "statement_list":
    for statement in ast["statements"]:
        evaluate(statement, environment)
    return None
```

Every call receives the same environment dictionary. Assignment mutates that dictionary, so later statements see the values created by earlier statements. If the evaluator created a fresh dictionary for every statement, `print x` would fail immediately after `x = 2`.

The program node delegates to its statement list:

```
elif ast["tag"] == "program":
    evaluate(ast["statements"], environment)
    return None
```

The program itself has no separate final numeric value. Its observable results are the output it produces and the changes made to its environment while it runs.

Following example.v

The complete execution can be traced one statement at a time:

Statement	Expression value	Environment afterward	Output
`x = 2`	`2`	`{"x": 2}`	
`print x`	`2`	`{"x": 2}`	`2`
`y = x + 3 * 4`	`14`	`{"x": 2, "y": 14}`	
`print y`	`14`	`{"x": 2, "y": 14}`	`14`

The arithmetic expression in the third statement still uses the Chapter 1 precedence rules. The parser groups it as `x + (3 * 4)`. The evaluator retrieves `2` for `x`, computes `12`, and then computes `14`. Assignment stores that result as `y` before the final print statement runs.

This trace shows the main reason for introducing a program node and a shared environment. Neither statement order nor stored values could be represented by the old model of one isolated expression.

The Runner

The runner connects the stages using an empty environment:

```
tokens = tokenize(program)
ast = parse(tokens)
evaluate(ast, {})
```

The same three interpreter stages remain:

Stage	Input	Output
Tokenizer	Program text	Token list
Parser	Token list	Program tree
Evaluator	Program tree and environment	State changes and output

The interfaces have expanded, but their responsibilities remain separate. The tokenizer recognizes names and punctuation. The parser determines statement structure. The evaluator gives assignment, lookup, sequencing, and print their behavior.

Real-World Tokenizers and Parsers

The interpreter in this course is small, but the work is not toy work in the sense of being unrelated to real language systems. Production compilers and interpreters still have to answer the same early questions:

- What characters form the next token?

- Is this word an identifier or a keyword?
- Is this number an integer, a floating-point literal, or something invalid?
- Which expression should bind first?
- Is this source text an expression, an assignment statement, or some other statement?

Go's standard library has a scanner in `go/scanner/scanner.go`. Its main scanning function skips whitespace, checks for identifiers and keywords, recognizes numbers, and then switches on punctuation and operators. (5) That is the same basic job as Vertex's tokenizer, just with many more cases:

```
case isLetter(ch):
    lit = s.scanIdentifier()
    tok = token.Lookup(lit)
case isDecimal(ch) || ch == '.' && isDecimal(rune(s.peek())):
    tok, lit = s.scanNumber()
```

Go's number scanner also has the same basic split we just added: start with the integer part, then handle a fractional part if a dot appears. The real scanner has to support more forms, including bases, underscores, and exponents, but the central idea is familiar:

```
if s.ch == '.' {
    tok = token.FLOAT
    s.next()
    digsep |= s.digits(base, &invalid)
}
```

The Go parser also has a function named `parseUnaryExpr`. (6) It recognizes unary operators, consumes the operator token, and recursively parses the operand:

```
case token.ADD, token.SUB, token.NOT, token.XOR, token.AND, token.TILDE:
    pos, op := p.pos, p.tok
    p.next()
    x := p.parseUnaryExpr()
    return &ast.UnaryExpr{OpPos: pos, Op: op, X: x}
```

That is the same pattern as Vertex's `parse_unary`: a leading minus sign creates a unary node whose operand is another unary expression or a factor. Go has more operators and richer AST node types, but the parsing move is the same.

The GNU Compiler Collection (GCC) shows the same ideas at a much larger scale. Its C lexer classifies tokens such as `CPP_NAME` and `CPP_NUMBER`. (7) Its C parser has functions such as `c_parser_unary_expression`, `c_parser_binary_expression`, and `c_parser_expression`. (8) Those files are much harder to read because GCC handles decades of C, extensions, diagnostics, optimization hooks, and target-specific concerns. Still, the front end begins with the same kind of work: recognize tokens, group them according to grammar rules, and build a representation that later compiler stages can use.

That is the useful comparison. Vertex is tiny so that the moving parts are visible. Real language systems are larger because their languages, error recovery, tooling, and compatibility promises are larger. They are not doing a completely different kind of work at the front end.

Errors and Tests

Run the Python tests from this chapter's code directory:

```
python3 tokenizer.py
python3 parser.py
python3 evaluator.py
```

The tokenizer tests cover the new operators, keyword, and identifiers. They also check comment handling, including line endings, end-of-input comments, following token positions, and ordinary division. Parser tests exercise each grammar rule directly and check complete programs. Evaluator tests check lookup, assignment, arithmetic with identifiers, and program execution.

Useful program-level cases include:

Input	Expected behavior	Stage to inspect
`x=2;print x`	Prints `2`	Complete pipeline
`x=-1.5;print x`	Prints `-1.5`	Float and unary minus
`x=3*-2;print x`	Prints `-6`	Unary operand in multiplication
`;;;x=2;;;print x;;;`	Prints `2`	Statement-list parsing
`x=2 print x`	Syntax error	Missing separator
`print x`	Unknown identifier error	Environment lookup
`print (x=2)`	Syntax error	Assignment remains a statement
empty input	Syntax error	Nonempty statement list
`print 3@4`	Unexpected-character error	Tokenization
`print 3/0`	Division-by-zero error	Evaluation

As in Chapter 1, a rejected invalid program is a successful result when the language rules require rejection. Tests should verify both accepted programs and the boundaries of the current grammar.

Considerations

1. What would break if each statement received a fresh environment instead of the same dictionary?
2. In `x = x + 1`, why must the right side be evaluated before replacing the stored value? What should happen if `x` has no value yet?
3. What ambiguity or additional rules would arise if newlines replaced semicolons as statement separators? Include comments in your example.
4. How would permitting assignment inside an expression change the grammar and the meaning of a program such as `print (x = 2)`?

Practice

Answers to the practice questions follow this section.

Work in a copy of the topic directory.

1. Run example.v. Before each statement executes, write down the environment you expect. Compare your trace with the output.
2. Tokenize `total = 3; print total`. Identify which rule recognizes each token and explain why `print` is not an identifier.
3. Use Python's pprint to inspect the complete AST for `x=2;print x`. Find the program node, statement-list node, and the two statement nodes.
4. Try the valid semicolon variations in the table above. Add parser tests for any forms not already covered.
5. Remove the semicolon from `x=2;print x`. Follow the unused tokens returned by parse_statement_list before the public parser reports the error.
6. Run `x=2; x=x+5; print x`. Trace which value of `x` the right side of the second assignment reads.
7. Run `x=-1.5; print x` and `x=3*-2; print x`. Identify which part of the grammar handles each minus sign.
8. Add an evaluator test that assigns two variables and uses both in a third assignment. Check the final environment as well as printed output.
9. Add a test that expects an unknown-identifier error. Explain why this is an evaluation error rather than a tokenization or syntax error.
10. Construct a print node manually and evaluate it with `{"answer": 42}`. Explain what this tests without involving the tokenizer or parser.
11. As an extension on paper only, propose grammar and AST changes for allowing a plain expression as a statement. Identify the parser decision that would become more complicated.

Answers to the Practice Questions

1. Before `x=2`, the environment is empty. Before `print x`, it contains `x: 2`. Before `y=x+3*4`, it still contains only that binding. Before `print y`, it also contains `y: 14`. Output is `2`, then `14`.

2. `total` is an identifier; `=` and `;` are punctuation tokens; `3` is a number; `print` is a keyword; the final `total` is another identifier. The keyword pattern precedes the identifier pattern, so the scanner recognizes `print` before the more general identifier rule can match it.

3. The program node contains a statement-list node. Its first statement is an assignment with name `x` and a number expression holding `2`. Its second is a print statement whose expression is an identifier node for `x`. The tree records syntax; it does not contain an environment value for `x` yet.

4. All of `x=2`, `x=2;`, `;;;x=2;;;`, `x=2;print x`, and `x=2;;;print x` parse. `x=2 print x` does not. Leading and trailing separators may repeat, but adjacent statements need at least one separator.

5. The statement-list parser consumes `x=2` and leaves the `print` and `x` tokens. Because no semicolon introduced another statement, that parser stops. The public parser sees unused tokens and raises `SyntaxError`.

6. The second assignment reads the existing value `2`, adds `5`, then stores `7` under the same name. It does not overwrite `x` before reading its right-hand side. The print produces `7`.

7. These print `-1.5` and `-6`. The `unary` grammar rule handles both minus signs, including the one following multiplication. Neither is part of a number token.

8. One test checks the final bindings and captured terminal output together:

```
import io
from contextlib import redirect_stdout
from tokenizer import tokenize
from parser import parse
from evaluator import evaluate

environment = {}
output = io.StringIO()
with redirect_stdout(output):
    evaluate(parse(tokenize("a=3;b=4;c=a+b;print c")), environment)
assert environment == {"a": 3, "b": 4, "c": 7}
assert output.getvalue() == "7\n"
```

9. `print missing` tokenizes and parses, but evaluation raises `ValueError` because `missing` has no binding. The identifier's spelling is legal; only the environment can answer whether a value exists for that name.

10. Evaluate a print node with an identifier expression for `answer` in `{"answer": 42}`. It prints `42`. This tests print and identifier lookup directly, without testing whether the source text can be parsed.

11. One possible extension is `statement ::= assignment_statement | print_statement | expression_statement`, with an expression-statement node storing its expression. An identifier could then begin either `x=2` or `x+2`, so choosing a statement rule would require looking ahead or parsing the shared beginning before deciding which form follows.

References

1. Wikipedia contributors. “Statement (computer science).” Wikipedia. https://en.wikipedia.org/wiki/Statement_(computer_science)
2. Wikipedia contributors. “Assignment (computer science).” Wikipedia. https://en.wikipedia.org/wiki/Assignment_(computer_science)
3. Wikipedia contributors. “Aliasing (computing).” Wikipedia. https://en.wikipedia.org/wiki/Aliasing_(computing)
4. Wikipedia contributors. “Extended Backus-Naur form.” Wikipedia. https://en.wikipedia.org/wiki/Extended_Backus%E2%80%93Naur_form
5. The Go Authors. Go standard library, `go/scanner/scanner.go`, `Scan` (abridged excerpt). https://go.dev/src/go/scanner/scanner.go
6. The Go Authors. Go standard library, `go/parser/parser.go`, `parseUnaryExpr` (abridged excerpt). https://go.dev/src/go/parser/parser.go
7. GNU Project. GNU Compiler Collection, `gcc/c-family/c-lex.cc`. https://github.com/gcc-mirror/gcc/blob/master/gcc/c-family/c-lex.cc
8. GNU Project. GNU Compiler Collection, `gcc/c/c-parser.cc`. https://github.com/gcc-mirror/gcc/blob/master/gcc/c/c-parser.cc